TL2 (Type Language 2) Primer

Таблица версий документа

01 июня 2024	Первая версия
03 июня 2024	Мелкие комментарии и добавления
30 июня 2024	Поменял раздел про upgrade полей
14 июля 2024	Мелкие уточнения и предложения
15 мая 2025	Число элементов массива сериализуем явно.
19 мая 2025	Отказываемся от magic совсем, используем полное имя метода, как идентификатор
28 декабря 2025	Изменение формат на основании feedback и опыта миграции сервисов. Разделение типов bool и бит, разделение алиаса и структуры с 1 полем. Тип словарь.
31 декабря 2025	Раздел, про большие и маленькие буквы. Много уточнений. С Новым Годом!
1 января 2026	Уточнения об опциональных полях типа bit, неуказанный magic равен 0, исправлены опечатки.

Общие сведения.....3
Краткий справочник..... 3
Грамматика.....4
Простые типы.....5
Численные типы..... 5
Строка..... 5
Bool и Bit..... 6
Структуры..... 6
Optional..... 8
bool..... 9
bit..... 9
true-тип (void).....10
Объединения (union).....10
Перечисления (Enum)..... 11
Детали синтаксиса объединений..... 12
Коллекции..... 12
Массивы..... 12
Словари..... 13
Функции (Function)..... 14
Typedef и Alias..... 15
Magic..... 16
Шаблонные типа.....16
#-параметры и явные маски полей.....16
type-параметры..... 17
Идеи на будущее.....17
InvokeReq-подобные обёртки.....17
Upgrade полей.....17
Переход с TL1 на TL2.....19

Общие сведения

TL2 это язык описания данных и формат хранения данных.

С практической точки зрения, TL2 описывает структуры данных, в том числе RPC запросы и ответы на них.

Формат TL2 призван решить проблемы оригинального формата TL1, не потеряв самое важное свойство - скорость чтения и записи. TL2 спроектирован так, чтобы занимать нишу между находящимися с одной стороны бинарным форматом и TL1 (нет никакой разметки, максимальная скорость сериализации, плохая совместимость), и находящейся с другой стороны BSON, MsgPack, ProtoBuf (много разметки, небольшая скорость сериализации, хорошая совместимость).

В этом документе много сравнений с TL1, если вы ничего не знаете о TL1, можно пропустить их при чтении.

TL2 это вектор развития, противоположный вектору развития проектов телеграмм (TLB). Если там идёт дальнейшее увеличение сложности, то в TL2 простота – основное требование

TL2-файл обычно даётся кодогенератору для того, чтобы тот сгенерировал определения структур данных для какого-либо языка программирования, и функции (де)сериализации этих данных, а также код RPC клиентов и серверов, обменивающихся сообщениями, сериализованными в соответствующем формате.

TL2 как и TL1 содержит некоторое количество встроенных простых типов, таких как int32, и конструкции, позволяющие комбинировать их в более сложные ***комбинаторы*** - ***структуры*** и ***объединения*** (вместе они называются ***объекты***), ***массивы***, ***словари*** и ***функции***.

Краткий справочник

```
// типы
user = id:int64 name:string; // структура
user = title?:string married:bool set:bit; // опциональность и bool
getUserResult = found user:user | notFound // объединение
color = red | green | blue; // перечисление
user = parents:[2]int64 children:[]int64; // массивы
user = visits:[string]int32 likes:[int64]bool; // словари
listID <=> int64; // алиас
// шаблоны
```

```
point<x:type, y:type> = x:x y:y;
tree<n:#, v:type> = chidren:[n]node v:v;
// функции, возвращающие
memcacheSet key:string value:string => ; // ничего
getUserID name:string => int64; // значение
getUser id:int64 => name:int64 group:uint64; // структуру
getUserName id:int64 => found name:string | notfound; // объединение
```

Грамматика

Идентификаторы в TL2 могут начинаться как с большой, так и с маленькой латинской (unicode basic latin) буквы, и содержать латинские буквы, цифры и знак подчёркивания.

Никакой смысловой нагрузки заглавность идентификаторов не несёт. Но ссылаться на тип можно только точно указав его имя.

При этом запрещены идентификаторы, отличающиеся от других только заглавностью букв и расположением знаков подчёркивания. Также запрещены идентификаторы, содержащие более одного знака подчёркивания подряд.

Эти запреты исходят из того, что кодогенератор TL2 может быть для разных языков, при этом, например, в golang заглавность идентификатора определяет его видимость. Кодогенератор хочет преобразовывать заглавность, а также генерировать суффиксы для имён без коллизий с другими идентификаторами.

В TL2 есть несколько встроенных типов, они считаются объявленными до первого пользовательского объявления.

Грамматика TL2 максимально контекстно-независима и упрощена относительно TL1.

В TL2 идентификаторы из разных контекстов (имена комбинаторов, имена полей, вариантов объединений, и категорий (type)) не конфликтуют друг с другом. Локальные ссылки (на тип шаблона) имеют приоритет над глобальными идентификаторами. Поэтому следующий пример законен.

```
type<type:type> = type type:type | nothing;
```

Простые типы

Численные типы

```
int32 uint32 float32   int64 uint64 float64   byte
```

Сериализуются в little endian представлении без каких-либо оптимизаций размера или маркеров, к примеру uint32(9) uint64(5) byte(1) будут записаны в таком виде.

```
[09 00 00 00] [05 00 00 00 00 00 00 00] [01]
```

Мы не хотим сразу делать типы int16, uint16. Если будет необходимость, сделаем позже. Также у нас нет знакового int8. В TL2 отказались от имён типов C++ int, long, которые использовались в TL1 в пользу явного указания битности в имени типа.

Floating-point значения записываются в обычном для x86 формате IEEE 754 binary little endian.

Строка

```
string
```

Записывается, как длина, за которой идут сами байты. Дополнение до кратного 4 размера, как в TL1, не применяется.

```
[08] [T  I  N  Y  _  S  T  R]
```

Формат длины строки называется ***varlen*** и имеет 3 представления - крошечное, среднее и огромное.
Числа от 0 до 253 включительно записываются в крошечном представлении - единственный байт и есть длина, например длина 5 будет записана, как

```
[05]
```

Числа от 254 до 2^16+253 записываются в среднем представлении - первый байт-маркер со значением 254, и затем число минус 254 в формате uint16 little endian, например число 260 будет записано, как.

```
[FE] [06 00]
```

Самое маленькое число в этом представлении, 254, будет записано, как

```
[FE] [00 00]
```

Числа, которые не помещаются в крошечное или среднее представление, используют огромное представление - первый байт-маркер со значением 255, и затем само число в формате uint64 little endian, например число 2^24-1 будет записано, как

```
[FF] [FF FF FF 00 00 00 00 00]
```

Огромное представление подходит и для записи чисел, которые можно записать в крошечном и среднем представлении, это неоптимально, но разрешено.

Формат varlen также применяется для записи размера объектов и номеров вариантов union, то есть тех чисел, которые имеют тенденцию быть небольшими.

Малое число представлений положительно влияет на скорость (де)сериализации, не особо увеличивая размер, поэтому используем всего 3 представления, а не 9, как sqlite4 и varint.

Bool и Bit

Это тоже простые типы, но чтобы понять их отличия, нам нужно сначала рассмотреть сериализацию объектов.

Структуры

Рассмотрим следующее определение

```
point = x:int32 y:int32;
```

Он описывает структуру point с двумя целыми полями.

Рассмотрим сериализацию значения point(5 1)

```
[09] [06] [05 00 00 00] [01 00 00 00]
```

Первый байт - размер сериализованного представления (тела) структуры в байтах.

Второй байт - маска полей. Перед каждыми 8 определёнными пользователем полями структура неявно содержит и сериализует 1 байт маски, по биту на поле. Установленный бит маски означает, что поле сериализовано, а неустановленный - что поле пропущено. Если при чтении поле, известное читателю отсутствует (не сериализовано), оно выставляется в default значение (ноль, пустая строка, пустой объект, и т.д.)

Например, сериализация значения point(5 0) point(0 1) и point(0 0) может быть такой

```
[05] [02] [05 00 00 00]
[05] [04] [01 00 00 00]
[01] [00]
```

В последнем случае маска равна нулю, поэтому её саму можно не сериализовывать, и тогда нулевая точка в оптимальном формате занимает ровно 1 байт.

```
[00]
```

Следующее представление нулевой точки тоже корректно, хотя и не оптимально.

```
[09] [06] [00 00 00 00] [00 00 00 00]
```

Так же как в TL, все поля в структуре сериализуются по-очереди в порядке объявления. Это позволяет значительно выиграть в скорости и компактности сериализаторов у MsgPack и Protobuf, где поля хранятся в произвольном порядке в виде пар (идентификатор поля, значение поля).

Рассмотрим добавление в структуру Point нового поля

```
point = x:int32 y:int32 z:int32;
```

Новая реализация успешно читает значения, сериализованные старой, потому, что для новых полей в масках полей биты равны нулю.

Старая же реализация успешно читает значения, сериализованные новой, где есть новые поля, потому, что все известные ей поля идут до неизвестных.

```
[0D] [0E] [00 00 00 00] [00 00 00 00] [00 00 00 00]
```

Поэтому если при чтении все известные поля считаны, а в теле структуры остались байты, они пропускаются, для чего используется явный сериализованный размер структуры

Когда поля больше не нужны, они не удаляются, а маркируются, как неиспользуемые, чтобы сохранить нумерацию полей в масках.

```
point = x:int32 _:int32 z:int32;
```

При чтении, если соответствующий бит маски поставлен, знание типа поля позволяет пропустить (прочитав и выбросив) его. Таким образом, на каждые 8 убранных полей в сериализованном представлении структуры навсегда остаётся 1 байт, что довольно недорого.

Формат сериализации структуры (количество байтов, байты) совпадает с форматом сериализации строки, так что любую структуру можно прочитать в строку, затем сохранить строку обратно, не сломав сериализацию полей структуры. Это удобно для различных прокси, которые не желают разбираться во внутренних форматах сообщений. Также это позволяет убирать определения ненужных типов, как только они перестают использоваться. Например если в определении

```
externalType = x:someType y:int32;
```

поле x больше не нужно, то без потери совместимости можно написать не только

```
externalType = _:someType y:int32;
```

но и, когда определение someType больше нигде не используется

```
externalType = _:string y:int32;
```

И удалить определения someType из файлов TL2. При чтение если поле x было сериализовано предыдущими версиями, оно будет пропускаться, как строка.

TODO - есть предложение позволять отмечать некоторые поля, как frozen. Для таких полей бит маски не заводится, они не могут стать optional. Сэкономит несколько наносекунд для типов наподобие Pair, где всегда 2 поля. Пока думаем отказаться от этой идеи.

Optional

Любое поле можно промаркировать, как опциональное.

```
point = x?:int32 y?:int32;
```

Опциональность, это свойство поля, а не типа, поэтому вопросительный знак ставится слева от двоеточия.

Формат сериализации не изменится, однако соответствующий бит маски полей будет использоваться теперь как признак того, что optional был в установленном состоянии.

Раньше, когда поле было обязательным, применялась оптимизацию, когда вместо записи нулевого значения бит маски полей выставлялся в 0, поле пропусказолсь, а при чтении нулевое значение восстанавливалось. Поэтому новая реализация примет нулевые поля, записанные старой, за неустановленные опциональные поля, к этому надо быть готовым. В остальном же при изменении опциональности полей структура продолжает нормально читаться.

bool

bool это на самом деле byte, для которого при генерации кода генерируется булевский тип. При чтении нулевое значение интерпретируется, как false, остальные, как true. При записи пишется 0 или 1. bool в любом контексте можно без потери совместимости превратить в byte и наоборот.

```
weather = sunny:bool showers:bool;
```

Например weather(true false) будет сериализовано, как

```
[02] [02] [01]
```

Здесь 02 это маска полей, которая говорит, что поле sunny установлено, а значение поля sunny это 01, то есть true. Поле showers было убрано (подвергнуто оптимизации) с помощью маски полей.

weather(true true) будет сериализовано, как

```
[03] [06] [01] [01]
```

Здесь маска 06 означает, что поля sunny и showers присутствуют.

Поля bool могут быть опциональными, как обычно. Как обычно же, если опциональное поле установлено, то бит маски будет установлен, отражая этот факт, а значение уже будет 00 или 01, в зависимости от значения bool.

```
weather = sunny?:bool showers?:bool;
```

Например weather с установленными и равными false полями будет сериализовано, как

```
[03] [06] [00] [00]
```

Иногда расширяемость и опциональность byte не нужна, а нужна компактность, поэтому в TL2 есть ещё один булевский тип – bit.

bit

Значения этого типа не занимают места, кроме бита маски полей.

```
Weather = sunny:bit showers:bit;
```

Например weather(true false) будет сериализовано, как

```
[01] [02]
```

А значение weather(true true), как

```
[01] [06]
```

Это аналогично трюку с ?true-типом в TL1.

Поскольку в имени bit уже заложено два значения, соответствующие биту маски полей, используемой в остальных случаях под признак опциональности, поля типа bit нельзя объявлять опциональными, даже если такое поле получилось при подстановке шаблонного типа.

```
placeholder<t:type> = x:?t;
usePlaceholder = a:placeholder<int32> b:placeholder<bit>;
```

В дальнейшем, по мере накопления опыта, мы можем пересмотреть это правило и начать в этом случае игнорировать признак опциональности, но пока что решили действовать осторожно и запретить этот случай.

true-тип (void)

В TL2 можно задать структуру без полей.

```
true = ;
```

В отличие от TL, сериализация такой структуры занимает 1 байт (сериализуется 1 нулевой байт размера объекта).

```
[00]
```

Причин задавать такую структуру в TL2 нет, во всех контекстах, где требуется структура, можно задать пустую структуру просто поставив точку с запятой.

Объединения (union)

Вы могли заметить, что нумерация полей в примерах выше при установке битов маски полей, начиналась с 1, а не с 0. Это потому, что любая структура неявно содержит нулевое поле - номер варианта, и с помощью него может быть превращена в union.

Поскольку у структуры единственный вариант с номером равным 0, соответствующий бит маски не ставится и номер варианта для структуры не сериализуется.

Рассмотрим следующий тип ответа на запрос

```
getUserResult = user:user;
```

Мы решили, что когда пользователь не найден, мы не хотим возвращать ошибку, а хотим возвращать специальный ответ, поэтому превратим его в объединение

```
getUserResult = found user:user | notFound;
```

При сериализации старой версия getUserResult номер варианта неявно сериализовывался, как 0, так как первый бит первой маски полей для структуры ставился в 0, поэтому при десериализации новой версией будет корректно определён и прочитан первый вариант объединения. При сериализации же новой версией варианта notFound номер варианта, будет сохранён, как 1.

Старая реализация при попытке прочитать значение notFound вернёт ошибку. Однако старая реализация будет корректно читать значение found в свою структуру.

Если какой-то вариант больше не нужен, совсем убирать его нельзя, так как изменится нумерация вариантов. Но его можно промаркировать, как удалённый.

```
getUserResult = found user:user | _ | reallyNotFound;
```

В TL2 номера вариантов сохраняются в формате varlen - обычно для номера варианта достаточно 1 байта.

При определении объединения можно не давать имя полю, если оно единственное, тогда его тип указывается вместо списка полей:

```
getUserResult = found user:user | notFound | wasFound int32;
```

При генерации кода варианта wasFound при поддержке в языке будет сгенерирован typedef вместо структуры.

Перечисления (Enum)

Если все варианты объединения без полей, то как и в TL1, генерируется перечисление..

```
color = red | green | blue;
```

Сериализация перечисления занимает 1 байт для первого варианта (размер 0, дальше неуказанное первое поле равно по-умолчанию 0) и 3 байта для остальных (размер, маска полей, равная 1 в знак того, что сериализован номер вариант, и сам номер варианта).

Перечисления расширяются до объединений аналогичным TL1 образом - добавлением поля в любую структуру.

Детали синтаксиса объединений

Имя варианта может быть любым и пересекаться с любыми именами, кроме имён вариантов этого объединения, например здесь вариант с именем int32 является typedef на int32.

```
getUserResult = found user:user | notFound | int32 int32;
```

В определении же

```
getUserResult = found user:user | notFound | int32;
```

Последний вариант это структура без полей с именем int32, к самому типа int32 не имеет никакого отношения.

При определении объединения разрешено использовать лидирующую вертикальную палку, это нужно для определение объединений на нескольких строках. и для определения объединений с 1 вариантом, что не влияет на формат TL2, но влияет на генерацию кода.

```
getUserResult =
  | found user:user
  | notFound;
getUserResult = | found user:user | notFound;
getUserResult = | found user:user;
```

Коллекции

Массивы

Массивы в TL2, также как и объекты, сериализуются в виде размер тела (varlen), байты тела. Если тело пустое, значит это пустой массив. В противном случае сначала указывается число объектов, тоже в формате varlen, затем идут сами объекты. Знание числа объектов позволяет заранее аллоцировать массив нужного размера, и занимает всего 1 дополнительный байт для коротких массивов.

Массивы бывают переменного и фиксированного размера.

```
brackets = a:[]int32 b:[3]int32;
```

При сериализации отличий нет - всегда явно записывается количество элементов. Так что при замене переменного массива на фиксированный и обратно, а также при изменении числа элементов фиксированного массива совместимость сохраняется. При

сериализации недостающие элементы будут установлены в default значения, а избыточные проигнорированы.

Рассмотрим сериализацию [3]int32 (либо []int32 с 3 элементами) содержащую значения [5, 6, 1]

```
[0d] [03] [05 00 00 00] [06 00 00 00] [01 00 00 00]
```

Вначале размер тела массива, затем тело - количество элементов (3) и сами элементы. Все элементы необходимо указывать, даже если они нулевые. Оптимизация, где суффикс нулевых элементов опускается, не применяется из опасения alloc amplification, когда можно было бы указать гигантскую длину массива в коротком сообщении.

Массивы с элементом типа bit имеют похожую, но специальную форму сериализации - в один байт пакуется 8 значений.

```
brackets = var:[]bit;
```

Рассмотрим сериализацию []bit{false,true,true,true}

```
[02] [04] [0E]
```

Здесь 4 это число элементов, для сериализации 4 элементов достаточно 1 байта, чьи биты выставлены соответствующим образом.

При этом массив bool имеет обычную форму сериализации, совпадающую с массивом byte

```
brackets var:[]bool;
```

Рассмотрим сериализацию []bool{false,true,true,true}

```
[05] [04] [00] [01] [01] [01]
```

Это менее компактно, но зато расширяется при необходимости до массива byte.

Массивы могут использоваться везде, где необходим тип, и могут содержать любые типы в качестве элементов.

```
moreBrackets = a:[][3]pair<[]int32,[3]int32>;
```

Словари

```
dictExample = a:[string]int32;
```

Здесь string это ключ, а int32 – значение. В отличие от массивов, словарей фиксированного размера не существует.

В TL2 нет отдельного бинарного формата для словарей, они сериализуются, как обычные массивы структур из 2 обязательных полей key, value. Определение выше имеет в точности такой же форма, как определение ниже, и они взаимозаменяемы без потери совместимости.

```
elem = key:string value:int32;
dictExample = a:[]elem;
```

Все простые типы, включая bit, можно использовать, как ключи.

```
dictExample = a:[string]bool b:[bit]bit;
```

Другие типы в качестве ключей не допускаются. Это связано со сложностью кодогенерации реализаций для разных языков, в разных языках есть разные ограничения на тип ключа словаря. Также при сериализации в JSON все простые типы можно сериализовать в ключ-строку, что не так с более сложными типами.

В качестве значений можно использовать вообще любые типы, как и в массивах.

```
dictExample = a:[string]pair<[4]byte,bit>;
```

При генерации кода словари могут представляться, как массивы пар, либо честные словари в соответствующем языке (map в golang). Реализация может выбрать любой порядок записи, но обычно реализация будет сортировать словарь перед сериализацией, если не установлена настройка не делать этого (детерминизм по-умолчанию). Также если реализация содержит массив пар с совпадающими ключами, при чтении в словарь все, кроме последней пары пропадут

Функции (Function)

Функция, это спецификация сериализации параметров некоторого RPC вызова вместе со спецификацией сериализации ответа.

```
getUser id:int32 => user;
```

Тело функции (слева от стрелочки) является обычной структурой и сериализуется по правилам структуры. (Тело функции не может быть объединением).

Результат функции всегда является объектом, даже если выглядит, как простой тип, массив или словарь.

```
getUserID name:string => int32;
```

На самом деле функция в обоих примерах выше возвращает объект с первым полем типа user и int32 соответственно. Так что при необходимости определение выше совместимым в обе стороны образом поменять на

```
getUserID name:string => id:int32 group:int32;
```

Также можно позже расширять возвращаемый тип до объединения.

```
getUserID name:string => found id:int32 group:int32 | notFound;
```

Если функция не возвращает ничего, она всё равно возвращает структуру без полей, которую потом можно расширить обычным образом.

```
setUserName id:int32 name:string => ;
```

Typedef и Alias

Можно сказать, что какой-то тип является полностью взаимозаменяемым с другим с помощью синтаксиса.

```
Height <=> int32;
myPoint <=> point;
```

Это именно полное соответствие, расширить тип-алиас (добавить поля) независимо от его типа реализации невозможно. При генерации кода обычно генерируется typedef в соответствующем языке.

Отдельный символ соответствия выбран, так как знак равенства определяет структуры с полями. Следующая запись при определении простых структур запрещена, нужно либо сделать алиас, либо дать имя первому полю структуры.

```
myPoint = point;
```

При определении Union и Function справа от знака равенства может находиться тип, однако на самом деле в этих контекстах всегда определяются структура с полями, а тип интерпретируется как тип первого поля.

```
getUserResult = found user | notFound;
getUser id:int32 => user;
```

В обоих случаях user здесь это тип первого поля - варианта union, либо объекта-результата функции.

Если тип ответа функции расширяется, и хочется дать этому типу имя, то перенести поля в новый тип и вернуть его из функции не получится. Так как теперь добавится ещё один уровень вложенности объектов - функция теперь возвращает объект с первым полем типа getUserIDResult, а не сам объект.

```
getUserIDResult = found id:int32 group:int32 | notFound;
getUserID name:string => getUserIDResult;
```

Поэтому при возврате значения функцией, если нужно убрать дополнительное оборачивание, разрешено использовать символ алиаса.

```
getUserIDResult = found id:int32 group:int32 | notFound;
getUserID name:string => <=>getUserIDResult;
```

В этом контексте тип-алиас должен быть объектом, остальные типы не разрешены.

Прежде чем критиковать этот синтаксис, стоит заметить, что мы ожидаем крайне редкое применение алиаса в результате функции.

Magic

Сам язык TL2 не нуждается в magic и не использует его ни в каких контекстах. Мы позволяем указывать magic функций и объектов, поскольку в VK при RPC вызовах функция идентифицируется по magic. Когда-нибудь мы перейдём к использованию просто имени функций для этой цели, но пока magic нужны, они считаются чем-то вроде аннотаций.

В отличие от TL2, если magic не указан, не делается никаких попыток назначить его автоматически. При этом явное указание нулевого magic запрещено, так что значение 0 означает, что он не был указан.

Шаблонные типа

#-параметры и явные маски полей

В TL2 нет синтаксиса для указания масок полей, каждое поле неявно ссылается на бит маски, добавляемой генератором.

Поэтому если #-параметры и передаются во вложенные объекты, то могут там использоваться только для указания размеров массивов. Основная мотивация оставить в

языке сложность, связанную с передачей #-параметров, это возможность кодогенератору генерировать в создаваемых структурах статические массивы (std::array в C++, array в golang, etc.) вместо динамических (std::vector в C++, slice в golang, etc.), что значительно уменьшает число аллокаций и увеличивает локальность данных.

```
tree<n:#> = nodes:[n]node;
```

При использовании tree нужно указать либо конкретное число, либо сослаться на другой #-параметр.

```
forest<n:#> = a:tree<n> b:tree<2>;
```

Фактически #-параметры имеют диапазон значений, поддерживаемых форматом varlen (64 бита), что более, чем достаточно.

type-параметры

Работают так же, как в TL

```
point<x:type, y:type> = x:x y:y;
```

Идеи на будущее

InvokeReq-подобные обёртки

Можно заметить, что как сама функция, так и её результат по формату являются структурой, то есть в сериализованном формате строкой. Поэтому в принципе можно оборачивать функции таким образом

```
invokeReq e:reqExtra tag:uint32 req:string
  => e:respExtra result:string;
```

После разворачивания обёртки десериализатор как запроса, так и ответа нормально отработает. Мы собираемся применять это свойство при модернизации заголовков RPC-вызовов, однако это обсуждение выходит за рамки языка TL2.

Upgrade полей

Есть некоторые изменения формата, которые мы ожидаем, но не хотим отражать в бинарном формате, так это значительно усложнит его и замедлит скорость работы.

Например, изменение поля с int32 до int64, или замена поля типа X на массив полей типа X, или замена поля типа X на структуру с первым полем типа X.

Для таких случаев можно сделать в генераторах наборы правил конверсии, и синтаксис, который отмечает предыдущую версию поля, как заброшенную, и создаёт новую версию поля. Например, определение

```
getUser id:int32 n:uint32 => user;
```

Можно поменять на

```
getUser ^id:int32 n:uint32 id:int64 => user;
```

С точки зрения бинарного формата оба поля с именем id никак не связаны. А вот в коде будет сгенерировано только поле нового типа int64, и если в сериализованном формате встретится старое поле, произойдет конверсия в новый формат. Сохраняться будет версия поля, помеченная птичкой, так что в нашем примере поле будет пока что сериализовано в старом формате, так что старые версии будут продолжать читать id из старого поля.

Когда все клиенты обновятся и будут знать новое определение поля, можно будет включить сериализацию поля в новом формате, пометив активным новое поле

```
getUser id:int32 n:uint32 ^id:int64 => user;
```

И, наконец, если в природе не останется данных, сохранённых предыдущей версий кода, то есть во всех сериализациях будет только новое поле, то можно будет окончательно забросить старое поле:

```
getUser _:int32 n:uint32 id:int64 => user;
```

Удалить его нельзя, как и любое другое поле, чтобы не сбилась нумерация полей в масках, и можно было пропускать и игнорировать старые версии поля.

Также в некоторых ситуация полезно попросить сериализовать сразу несколько версий поля, пометив несколько версий птичкой.

```
getUser ^id:int32 n:uint32 ^id:int64 => user;
```

В любом случае, если некоторое имя поле используется в структуре более одного раза, они должны иметь разный тип, и хотя бы одно из них должно быть помечено птичкой. Это сделано для исключения опечаток.

Переход с TL1 на TL2

Сейчас имеется тысячи определений TL1 которыми пользуются различные сервисы. Мы хотим постепенный переход с TL1 на TL2, сервер получает поддержку TL2, сохраняя поддержку TL1, затем появляются клиенты, использующие TL2, и наконец клиентов, использующих TL1 не остаётся.

На время переходного периода мы хотим генерировать код сериализации TL2 из комбинаторов TL1. Затем, когда сериализацию TL1 больше никто не использует, можно автоматически трансформировать комбинаторы в TL2 таким образом, чтобы формат сериализации не поменялся..

Пока этого не сделано, развитие комбинаторов ограничено правилами TL1.

Маски полей TL1 могут содержать пользовательские данные, поэтому при сериализации старые маски полей интерпретируются как обычные поля. Скрытые маски полей TL2 поддерживаются в синхронизированном состоянии.

Поля, зависящие от масок полей (внешних либо внутренних) становятся optional. После перехода на синтаксис TL2 это можно будет подкорректировать.

Поля вида x:mask.1?true превращаются в bit, Bool превращается в bool, vector<Bool> превращается в vector<bit>.

Maybe остаётся Maybe, замена на optional возможна не во всех контекстах (пример []maybe<int32>), а замена в половине контекстов скорее всего сломала бы много пользовательского кода.